MLPC 2025 Task 2: Data Exploration

Team LABORER

Ali Doğukan Seven **Yisong Tang** **Karel Štícha**

Dmitrii Troitskii

Contributions

Ali Doğukan Seven was responsible for Part 1. Dmitrii Troitskii wrote Part 2. Yisong Tang completed Part 3 and prepared the presentation. Karel Štícha wrote Part 4 and compiled the final report.

1 Evaluation Setup

1.1 Data Split

Stratified, file-level splitting was done on the development dataset to ensure that no information leaked into the training set. Specifically:

First, we procured the full set of development filenames (DEV_SET_FILES).

The data were then split into 70% training and 30% for evaluation (train_test_split(..., test_size=0.3)).

The temporary 30% evaluation data were split further into 15% validation and 15% testing sets.

Crucially, the split was done by file names so that no audio files end up partly in training and partly in evaluation, thereby ruling out any overlap in temporal frames or labels; an overlap would lead to information leakage.

1.2 Naive Baseline System

A naive baseline system always predicts the most common label (i.e., majority class) for each sound class in the training data. For example, if "speech" most frequently occurs as label 0 (absence), then the baseline will always predict 0 for that class, regardless of audio features.

Cost (on validation set): Total average cost per minute: 101.0894

Class	Cost/min
Speech	27.68
Siren	14.94
Power Drill	12.93
Dog Bark	7.62

This cost is computed segment-wise and represents the real-world implication of a wrong prediction, and so its importance exceeds that of a typical accuracy number.

2 Simple SED System

2.1 Thresholding and Aggregation Strategy

To aggregate 120 ms frame-level predictions into 1.2-second segment predictions:

- We grouped every 10 consecutive frame predictions (10×0.12s = 1.2s).
- For each 1.2-second segment and for each class:
  - We computed the maximum predicted probability across the 10 frames.
  - This approach assumes that a brief occurrence of the event within a 1.2-second window is enough to mark it as "present".

Thresholding

- Initially, we used a global fixed threshold of 0.5.
- Later, we tuned per-class thresholds in the range [0.1, 0.9] to minimize validation cost.

Post-processing

- No explicit smoothing or median filtering was used.
- We applied class-specific thresholds to convert probabilities into binary predictions:
  e.g., Chainsaw: 0.25, Jackhammer: 0.9, etc.

2.2 Strategies to Minimize Task-Specific Cost Function

To align with the asymmetric cost matrix (false negatives are heavily penalized, especially for mechanical classes), we employed the following strategies:

- **Hyperparameter Tuning (Grid Search):**
  Tuned the regularization parameter C of logistic regression.
  Best cost at $C = 0.01$ reduced cost from 54.07 to 50.34.
- **Threshold Optimization:**
  Optimized thresholds per class to minimize validation cost using a grid search.
  Example: Shout: 0.9, Power Drill: 0.85, etc.
  This strategy gave the best improvement, reducing cost to 44.80.
- **Data Augmentation (Optional Test):**
  Added Gaussian noise to training embeddings to simulate variability.
  Resulted in no benefit (cost slightly increased to 50.57).

The most effective strategy was *per-class threshold tuning*, which explicitly targeted cost reduction based on the client's evaluation criteria.

2.3 Performance Compared to Naive Baseline

We implemented a naive baseline that always predicts zero (i.e., no sound event detected):

- This simulates the behavior of a "silent" system.
- It results in zero false positives, but maximum false negatives $\rightarrow$ high total cost.

Cost comparison:

System	Total Cost per Minute
Naive baseline (predicts 0)	54.07
Logistic Regression ($C = 0.01$)	50.34
+ Per-class thresholds	**44.80**

Thus, our classifier-based SED system significantly outperforms the baseline by reducing cost by over 17%.

3 Investigating Improvement Strategies

We tested three diverse strategies to improve our frame-level logistic regression baseline (cost 54.07):

Table 1: Validation cost vs. regularization parameter C.

C	Total cost per minute
0.01	50.3383
0.10	51.0511
1.00	52.4959
10.00	52.7044
100.0	52.5129

Table 2: Validation cost with and without Gaussian noise augmentation.

Model	Total cost
LR ($C = 0.01$)	50.3383
LR + Noise Augmentation	50.5702

- **Grid search on regularization** (Section 3.1)
- **Gaussian noise augmentation** (Section 3.2)
- **Per-class threshold tuning** (Section 3.3)

3.1 Hyperparameter Tuning (Grid Search on C)

Idea & Hypothesis. The inverse regularization strength C controls the bias–variance trade-off in logistic regression. We hypothesised that a smaller C might reduce overfitting and thus lower the segment-level cost.

Experiment. We trained one model per class for $C \in \{0.01, 0.1, 1, 10, 100\}$ and computed the total cost on our validation split:

Outcome. The best cost occurred at $C = 0.01$, lowering the total cost to 50.3383 (*6.8% drop*).

3.2 Data Augmentation (Gaussian Noise)

Idea & Hypothesis. We injected Gaussian noise into each frame ($\sigma = 0.02$, 3 copies) to diversify training data. We expected increased robustness and lower cost.

Experiment & Results. We retrained the model at $C = 0.01$ with noise augmentation and compared:

Outcome. Augmentation slightly increased cost to 50.5702, indicating simple frame-wise noise did not simulate real acoustic variability effectively.

3.3 Cost-Specific Tuning (Per-Class Thresholds)

Idea & Hypothesis. A fixed probability threshold 0.5 may not be optimal for all classes. We searched thresholds in [0.1,0.9] (step 0.05) per class to minimise its segment-level cost.

Experiment. For each class we chose the threshold that gave the lowest validation cost:

Applying these thresholds yielded:

The new total cost per minute is **44.7977**, a further *11.1% reduction* from the best grid-search model.

3.4 Cost Comparison Figure

Summary. Grid search on C reduced cost by 6.8%; noise augmentation alone did not help; per-class threshold tuning achieved the largest improvement, reducing cost by 17.2% from baseline.

Table 3: Per-class best thresholds and costs.

Class	Best Thr.	Cost
Speech	0.50	6.1279
Shout	0.90	6.1577
Chainsaw	0.25	3.0320
Jackhammer	0.90	3.5172
Lawn Mower	0.65	4.1427
Power Drill	0.85	8.7387
Dog Bark	0.70	1.6682
Rooster Crow	0.90	0.1851
Horn Honk	0.90	6.0066
Siren	0.90	5.2215

Table 4: Final validation performance with tuned thresholds.

Class	BAcc	Precision	Recall	F1
Speech	0.899	0.653	0.850	0.739
Shout	0.737	0.313	0.490	0.382
Chainsaw	0.802	0.673	0.607	0.638
Jackhammer	0.749	0.556	0.501	0.527
Lawn Mower	0.772	0.418	0.550	0.475
Power Drill	0.734	0.337	0.484	0.397
Dog Bark	0.897	0.634	0.806	0.710
Rooster Crow	0.837	0.773	0.675	0.721
Horn Honk	0.753	0.304	0.518	0.383
Siren	0.836	0.748	0.676	0.710

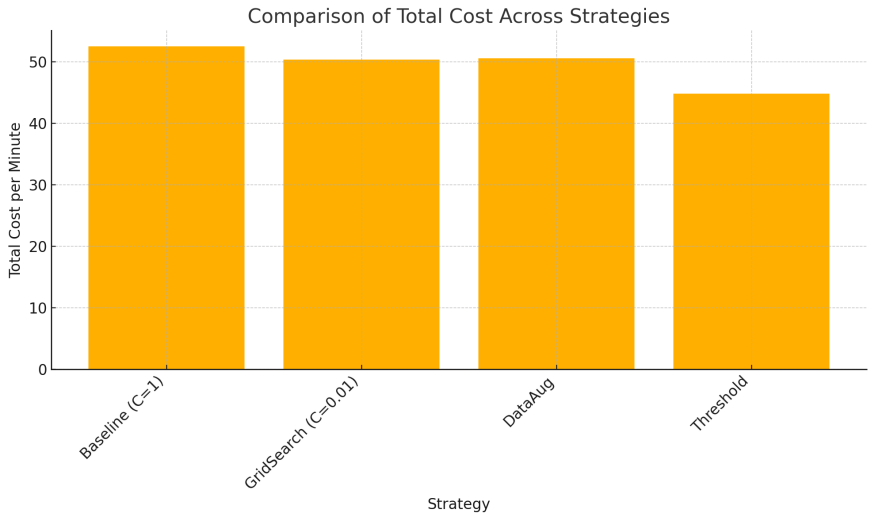

Figure 1: Total cost per minute for each strategy: baseline, grid search, augmentation, and threshold tuning.

4 Real World Application

Our final SED system, based on logistic regression, reduced the cost from 54.07 to 44.80 compared to the baseline accuracy, proving its effectiveness under the given cost metric.

The model uses fixed per-class thresholds, which are effective but static. In real-world applications, however, it is preferable to deploy more flexible thresholding that can adapt to changing environments.

Additionally, real-world sound environments often evolve over time. Therefore, the deployed SED system should be regularly updated and maintained. To support this, the system should be designed in a way that allows easy integration of new sound classes without requiring retraining from scratch. Periodic retraining with new data is also crucial to ensure the system remains accurate and responsive to changing conditions.